COLLEGE HELP CHATBOT

A Rule-Based Chatbot Using Python If-Else Conditions

Project domain	College information and student support
Core technology	Python, if / elif / else conditions
Interfaces	Terminal chatbot and Streamlit web interface
External AI service	Not required

PROJECT REPORT

Prepared as an academic demonstration project

1. Introduction

A rule-based chatbot is a program that responds to user messages by checking for predefined keywords or patterns. This project uses the college domain because students commonly need quick guidance about admissions, courses, fees, exams, attendance, library services, scholarships, hostel facilities, transportation, and placement support.

The chatbot is implemented in Python with explicit if/elif/else conditions. It does not train a model, call an external AI API, or require an internet connection for the terminal version. A Streamlit interface is included to make the same rules available through a browser.

1.1 Objectives

• Build a simple and understandable chatbot using conditional statements.

• Provide relevant replies to a range of college-related questions.

• Handle greetings, help requests, thanks, exits, blank input, and unknown questions.

• Offer both a command-line interface and a browser-based interface.

• Demonstrate the program with sample conversations and basic automated tests.

1.2 Scope and assumptions

The chatbot provides general guidance rather than official institution-specific data. It does not claim exact fee values, class hours, dates, telephone numbers, or eligibility decisions because these details vary by college and academic year. The user is directed to the official college website or responsible office for confirmation.

2. Features and Topic Coverage

Topic	Example user question	Response purpose
Admissions	How do I apply?	Directs students to official requirements and application dates.
Courses	What courses are available?	Suggests checking the official prospectus for the exact list.
Fees	How can I check tuition fees?	Explains that fees depend on program and year.
Exams and results	Where is the exam timetable?	Points to the examination portal or notice board.
Attendance and leave	How do I check attendance?	Advises checking the student portal and official leave process.
Library	How can I borrow a book?	Explains where to check borrowing and due-date rules.
Scholarships	Are scholarships available?	Directs the user to the scholarship office and official scheme.
Hostel / transport	How do I apply for a hostel?	Recommends the warden or transport office for current details.
Placements	Are internships available?	Directs to placement or career services.

3. System Design and Methodology

3.1 Input-processing flow

The user enters a message. The program converts it to lowercase and removes extra spaces. It checks the message against a sequence of keyword rules. When a condition matches, the related reply is returned. If no condition matches, the fallback rule asks the user to choose a supported topic.

1. Read user message
2. Normalize to lowercase and trim spaces
3. Test ordered if / elif rules
4. Return matching topic response
5. If nothing matches, return fallback response

3.2 Rule-based algorithm

• Start the program and display a welcome message.

• Read the user message from the terminal or web chat input.

• If the message is blank, request a valid question.

• Else if it matches greeting, admissions, courses, fees, exams, or another supported topic, return that topic response.

• Else if it is an exit command, end the terminal session.

• Otherwise return the unknown-question response.

• Repeat until the user exits the terminal session.

3.3 Main logic example

```
message = user_input.lower().strip()

if not message:
    return "Please type a question about college..."
elif "admission" in message or "apply" in message:
    return "Check the official admission office or website..."
elif "fee" in message or "fees" in message:
    return "Check the official fee circular or accounts office..."
else:
    return "Sorry, I do not have a rule for that question yet."
```

The actual chatbot.py file contains more topic rules and all response text. Rules are checked from top to bottom, so order matters when keywords could overlap.

3.4 Project files

chatbot.py contains the core reply rules and terminal loop. **app.py** provides the Streamlit browser interface. **test_chatbot.py** contains basic unit tests. **requirements.txt** lists the web dependency, while **README.md** explains setup, testing, GitHub publication, and deployment.

4. Sample Chatbot Conversations

The following examples demonstrate how messages trigger different conditional rules. Wording below is taken from the implemented chatbot responses.

Conversation 1: Greeting and admission

User: Hello

Bot: Hello! Welcome to College Help Chatbot. I can answer general questions about admissions, courses, fees, timings, exams, attendance, library, scholarships, hostel, transport, and placements. What would you like to know?

User: How do I apply for admission?

Bot: Admissions: Check your college's official website or contact the admission office for eligibility, application dates, required documents, and the admission form. Requirements can vary by course and academic year.

Conversation 2: Fees enquiry

User: Tell me about tuition fees

Bot: Fees: The amount depends on your course, category, and academic year. Please check the official fee circular or contact the accounts office. Ask for the fee structure, due dates, accepted payment methods, and receipt process.

Conversation 3: Library enquiry

User: How can I borrow a book?

Bot: Library: Visit the college library or library portal to search for books, check borrowing limits, renew loans, and confirm opening hours. Return books by the due date to avoid any applicable late fine.

Conversation 4: Unsupported question

User: Can you order pizza?

Bot: Sorry, I don't have a rule for that question yet. Try asking about admissions, courses, fees, timings, exams, attendance, library, scholarships, hostel, transport, placements, or contact details.

Conversation 5: Goodbye

User: bye

Bot: Goodbye! Best wishes for your studies. Type exit in the terminal to close the chatbot.

5. Testing and Execution

5.1 Basic test cases

Test	Input	Expected behavior	Status
TC-01	hello	Greeting response	Pass
TC-02	How do I apply for admission?	Admissions guidance	Pass
TC-03	tuition fees	Fees guidance	Pass
TC-04	Can you order pizza?	Fallback response	Pass
TC-05	spaces only	Request valid question	Pass

Test cases are automated in test_chatbot.py. The status column above describes the included assertions; the user can rerun them in their own Python environment using the command below.

5.2 Run the terminal chatbot

```
python chatbot.py
```

No third-party package is needed to run chatbot.py. Python 3.10 or newer is recommended.

5.3 Run the browser interface

```
python -m pip install -r requirements.txt
streamlit run app.py
```

The Streamlit page opens locally at the address printed in the terminal, usually http://localhost:8501. The sidebar contains quick-question buttons and a control to clear the conversation.

5.4 Run tests

```
python -m unittest -v test_chatbot.py
```

5.5 Evaluation

The design is easy to read, runs locally, and makes its rules explicit. Its accuracy depends on the keywords written by the developer. Similar phrases that do not contain a known keyword may reach the fallback response. Current college-specific details should be added only after verification with an official source.

6. GitHub and Live Deployment

The project files are arranged so they can be published to GitHub and deployed as a public Streamlit application. Actual public links are created only after publishing to a GitHub account and completing the Streamlit deployment; they cannot be truthfully provided before those steps.

6.1 GitHub repository steps

• Sign in to GitHub and create a new repository named college-help-chatbot.

• Upload chatbot.py, app.py, test_chatbot.py, requirements.txt, README.md, and the report PDF.

• If using Git from a terminal, run git init, git add ., git commit -m "Add college help chatbot project", and push the main branch to the new repository.

• Replace the sample YOUR-USERNAME in the README with the actual GitHub account name.

Repository URL pattern (example only): https://github.com/YOUR-USERNAME/college-help-chatbot

6.2 Streamlit Community Cloud deployment

• Open Streamlit Community Cloud and sign in using the GitHub account containing the repository.

• Choose Create app, then select the repository, main branch, and entrypoint app.py.

• Optionally set a custom subdomain and choose Deploy.

• After deployment succeeds, copy the real app URL displayed by Streamlit. It uses the streamlit.app domain.

Deployment guide: Official Streamlit Community Cloud deployment instructions

New repository page: Create a GitHub repository

Streamlit workspace: Streamlit Community Cloud

7. Limitations and Future Scope

Limitations

• Replies are predefined and do not understand meaning beyond the coded keywords.

• The demo has no database, user accounts, college timetable integration, or live updates.

• Exact college-specific data is intentionally not fabricated.

Future improvements

• Add a verified FAQ dataset for a particular college and academic year.

• Support multiple languages and spelling variations.

• Add a database or official API connection for current notices and schedules.

• Collect user feedback and expand the rule set based on frequently asked questions.

8. Conclusion

The College Help Chatbot demonstrates the fundamentals of a rule-based conversational program using Python if/elif/else conditions. It covers common student-service topics, handles unknown questions safely, and includes a browser interface for easier interaction. Sample conversations and basic tests illustrate its behavior. The project is suitable as a beginner-level academic submission and can be expanded with verified college information in future versions.

References

1. Python documentation: https://docs.python.org/3/ (conditional statements and input/output).

2. Streamlit documentation: https://docs.streamlit.io/ (web interface and deployment).

3. Streamlit Community Cloud deployment guide: https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/deploy.

Project publication status at report creation: code package prepared; GitHub repository and live hosting are pending account publication/deployment.